

Funkcje okna



LAG

Funkcje **LAG()** i **LEAD()** to funkcje okienkowe w SQL, które umożliwiają pobieranie wartości z poprzedniego lub następnego wiersza w zestawie wyników, zgodnie z określonym porządkiem. Oto, do czego służą:

- **LAG():**

Umożliwia pobranie wartości z wiersza poprzedzającego bieżący wiersz. Przydatna jest np. do porównywania bieżącej wartości z wartością z poprzedniego okresu lub rekordu, co umożliwia obliczanie różnic czy zmian.

- **LEAD():**

Umożliwia pobranie wartości z wiersza następującego po bieżącym wierszu. Dzięki temu można sprawdzić, jaką wartość ma kolejny rekord względem bieżącego, co też jest użyteczne przy analizach porównawczych i prognozowaniu trendów.



LAG

	last_name	hire_date	poprzednik
1	King	1987-06-17	---
2	Whalen	1987-09-17	King
3	Kochhar	1989-09-21	Whalen
4	Hunold	1990-01-03	Kochhar
5	Ernst	1991-05-21	Hunold
6	De Haan	1993-01-13	Ernst
7	Mavris	1994-06-07	De Haan
8	Baer	1994-06-07	Mavris
9	Higgins	1994-06-07	Baer
10	Gietz	1994-06-07	Higgins
11	Faviet	1994-08-16	Gietz
12	Greenberg	1994-08-17	Faviet

SELECT

last_name,

hire_date,

LAG(last_name,1,'----') OVER (ORDER BY hire_date) AS
poprzednik

FROM employees

LAG - partitional

	first_name	last_name	department_id	hire_date	previously
1	Kimberely	Grant	NULL	1999-05-24	-----
2	Jennifer	Whalen	10	1987-09-17	-----
3	Michael	Hartstein	20	1996-02-17	-----
4	Pat	Fay	20	1997-08-17	Michael Hartstein
5	Den	Raphaely	30	1994-12-07	-----
6	Alexander	Khoo	30	1995-05-18	Den Raphaely
7	Sigal	Tobias	30	1997-07-24	Alexander Khoo
8	Shelli	Baida	30	1997-12-24	Sigal Tobias
9	Guy	Himuro	30	1998-11-15	Shelli Baida
10	Karen	Colmenares	30	1999-08-10	Guy Himuro
11	Susan	Mavris	40	1994-06-07	-----

SELECT

first_name,
last_name,
department_id,
hire_date,

CASE LAG(CONCAT(first_name, ' ', last_name),1,0) OVER(PARTITION by department_id
ORDER BY hire_date
) WHEN '0' THEN '-----'
ELSE LAG(CONCAT(first_name, ' ', last_name),1,0) OVER(PARTITION by department_id
ORDER BY hire_date
)END AS previously

FROM

employees



LEAD

	last_name	hire_date	nastepny
1	King	1987-06-17	Whalen
2	Whalen	1987-09-17	Kochhar
3	Kochhar	1989-09-21	Hunold
4	Hunold	1990-01-03	Ernst
5	Ernst	1991-05-21	De Haan
6	De Haan	1993-01-13	Mavris
7	Mavris	1994-06-07	Baer
8	Baer	1994-06-07	Higgins
9	Higgins	1994-06-07	Gietz
10	Gietz	1994-06-07	Faviet

```
SELECT
last_name,
hire_date,
LEAD(last_name,1,'----') OVER (ORDER BY hire_date) AS nastepny
FROM employees
```

LEAD – partitional

	last_name	hire_date	department_id	nastepny
1	Grant	1999-05-24	NULL	---
2	Whalen	1987-09-17	10	---
3	Hartstein	1996-02-17	20	Fay
4	Fay	1997-08-17	20	---
5	Raphaely	1994-12-07	30	Khoo
6	Khoo	1995-05-18	30	Tobias
7	Tobias	1997-07-24	30	Baida
8	Baida	1997-12-24	30	Himuro
9	Himuro	1998-11-15	30	Colmenares
10	Colmenares	1999-08-10	30	---

```
SELECT
last_name,
hire_date,
department_id,
LEAD(last_name,1,'----') OVER (partition by department_id
                                ORDER BY hire_date) AS nastepny
FROM employees;
```



STRING_AGG

Funkcja **STRING_AGG** to funkcja agregująca w SQL, która umożliwia łączenie (konsolidację) wielu wartości tekstowych z kolumny w jeden ciąg znaków. W trakcie agregacji między poszczególnymi wartościami można wstawić określony separator, co ułatwia prezentację danych w postaci listy lub zestawienia.

Kluczowe zastosowania i cechy:

- **Agregacja tekstu:** Umożliwia złączenie wielu wierszy w jeden ciąg tekstowy, co jest przydatne np. przy tworzeniu listy elementów, tagów czy nazw.
- **Określony separator:** Użytkownik może zdefiniować separator, który będzie wstawiany między poszczególnymi wartościami (np. przecinek, średnik, spacja).
- **Łatwość użycia:** Pozwala uniknąć konieczności stosowania bardziej skomplikowanych rozwiązań, takich jak pętle czy podzapytania do konsolidacji danych.

STRING_AGG

	department_id	nazwiska
1	NULL	Grant
2	10	Whalen
3	20	Hartstein,Fay
4	30	Raphaely,Khoo,Baida,Tobias,Himuro,Colmenares
5	40	Mavris
6	50	Taylor,Fleaur,Sullivan,Geoni,Sarchand,Bull,Dellin...
7	60	Hunold,Emst,Austin,Pataballa,Lorentz
8	70	Baer
9	80	Johnson,Russell,Partners,Errazuriz,Cambrault,Zlo...
10	90	King,Kochhar,De Haan
11	100	Greenberg,Faviet,Chen,Sciarra,Uman,Popp
12	110	Higgins,Gietz

```
SELECT
    department_id,
    STRING_AGG(last_name,',') nazwiska
from employees
group by department_id
```


STRING_AGG - sortowanie

	job_id	nazwiska
6	FI_ACCOUNT	Chen,Faviet,Popp,Sciarra,Uman
7	FI_MGR	Greenberg
8	HR_REP	Mavris
9	IT_PROG	Austin,Ernst,Hunold,Lorentz,Pataballa
10	MK_MAN	Hartstein
11	MK_REP	Fay
12	PR_REP	Baer
13	PU_CLERK	Baida,Colmenares,Himuro,Khoo,Tobias
14	PU_MAN	Raphaely
15	SA_MAN	Cambrault,Erazuriz,Partners,Russell,Zlotkey
16	SA_REP	Abel,Ande,Banda,Bates,Bernstein,Bloom,...
17	SH_CLERK	Bell,Bull,Cabrio,Chung,Dellinger,Dilly,Ever...
18	ST_CLERK	Atkinson,Bissot,Davies,Gee,Ladwig,Landr...
19	ST_MAN	Fripp,Kaufling,Mourgos,Vollman,Weiss

```
SELECT  
job_id,  
STRING_AGG(last_name,',') within group (order by last_name) nazwiska  
from employees  
group by job_id
```

STRING_SPLIT

STRING_SPLIT to funkcja o wartości tabelarycznej, która dzieli ciąg znaków na wiersze podciągów na podstawie określonego znaku separatora

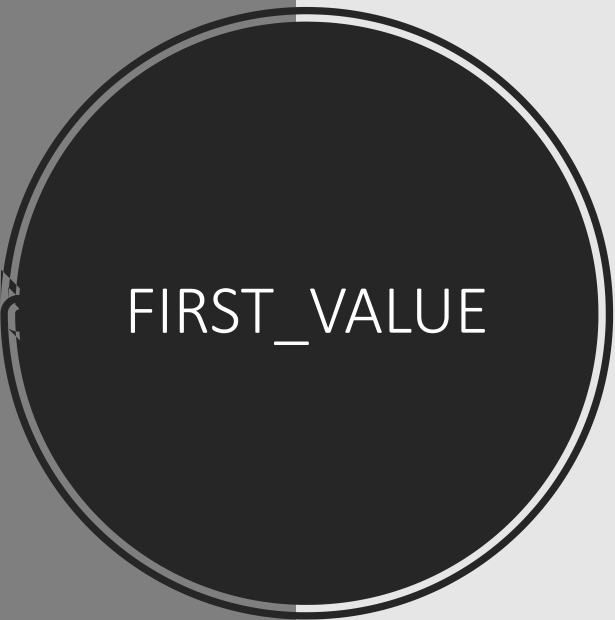
```
create table t1 (id int, tekst varchar(100))

insert into t1 values (1, 'abc,opo,ddd'), (2, 'xxx,ccc')

select * from t1

SELECT
    ID,
    TRIM(value) AS opis
FROM t1
CROSS APPLY STRING_SPLIT(tekst, ',');
```

	ID	opis
1	1	abc
2	1	opo
3	1	ddd
4	2	xxx
5	2	ccc



FIRST_VALUE

Funkcja **FIRST_VALUE()** to funkcja okienkowa w SQL, która zwraca pierwszą wartość z określonego zbioru wierszy w ramach okna (partycji), według ustalonego porządku sortowania. Dzięki niej można uzyskać wartość odniesienia dla danej grupy, co jest przydatne m.in. przy:

- **Porównaniach wartości:** Możesz porównać bieżącą wartość z pierwszą wartością w danej grupie, aby np. obliczyć różnicę czy procentową zmianę.
- **Ustaleniu wartości bazowej:** Umożliwia wyodrębnienie "pierwszego" rekordu, który może służyć jako punkt odniesienia w analizach.
- **Raportowaniu:** Pozwala na łatwe przedstawienie najwcześniejszej lub minimalnej (przy odpowiednim sortowaniu) wartości w ramach każdej partycji.

FIRST_VALUE

	department_id	last_name
1	NULL	Grant
2	10	Whalen
3	20	Hartstein
4	30	Raphaely
5	40	Mavris
6	50	Fripp
7	60	Hunold
8	70	Baer
9	80	Russell
10	90	King
11	100	Greenberg
12	110	Higgins

```
SELECT
distinct department_id,
FIRST_VALUE(last_name) over (partition by department_id
order by salary DESC) last_name
from employees;
```



LAST_VALUE

Funkcja **LAST_VALUE()** to funkcja okienkowa w SQL, która zwraca ostatnią wartość z określonego zbioru wierszy w obrębie zdefiniowanego okna (partycji), według ustalonego porządku sortowania.

Kluczowe aspekty:

- **Wyodrębnianie ostatniej wartości:**

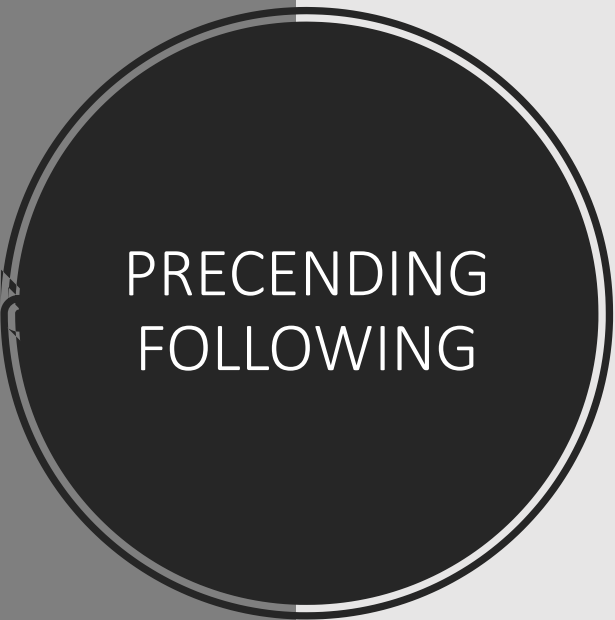
Funkcja ta pozwala pobrać wartość z ostatniego wiersza w danej partycji, co jest przydatne przy porównywaniu wyników lub wyciąganiu wartości końcowej z szeregu danych.

- **Definicja okna:**

Podobnie jak w przypadku innych funkcji okienkowych, do LAST_VALUE() używamy klauzuli OVER z opcjonalnym PARTITION BY oraz ORDER BY.

Należy jednak pamiętać, że domyślnie okno może być zdefiniowane jako "od początku partycji do bieżącego wiersza". Aby uzyskać wartość ostatniego wiersza całej partycji, warto jawnie określić zakres okna, np.:

ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING



PRECEDING FOLLOWING

Operatory **PRECEDING** i **FOLLOWING** są używane w klauzuli określającej ramy okna (ang. window frame) w funkcjach okienkowych SQL. Umożliwiają one zdefiniowanie zakresu wierszy, na których mają zostać wykonane obliczenia względem bieżącego wiersza, zgodnie z kolejnością określoną w klauzuli ORDER BY.

Kluczowe informacje:

- **PRECEDING:**
 - Odnosi się do wierszy, które występują **przed** bieżącym wierszem.
 - Można określić liczbę wierszy, np. *3 PRECEDING* oznacza trzy wiersze poprzedzające bieżący.
 - Specjalna wartość *UNBOUNDED PRECEDING* oznacza wszystkie wiersze od początku partycji aż do bieżącego wiersza.
- **FOLLOWING:**
 - Odnosi się do wierszy, które występują po bieżącym wierszu.
 - Można określić liczbę wierszy, np. *2 FOLLOWING* oznacza dwa wiersze następujące po bieżącym.
 - Specjalna wartość *UNBOUNDED FOLLOWING* oznacza wszystkie wiersze od bieżącego wiersza do końca partycji.

LAST_VALUE

	department_id	last_name
1	NULL	Grant
2	10	Whalen
3	20	Fay
4	30	Colmenares
5	40	Mavris
6	50	Olson
7	60	Lorentz
8	70	Baer
9	80	Kumar
10	90	De Haan
11	100	Popp
12	110	Gietz

```
SELECT
distinct department_id,
LAST_VALUE(last_name) over (partition by department_id
                             order by salary DESC RANGE BETWEEN
                             UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) last_name
from employees
order by department_id;
```

WINDOW declaration

	department_id	suma
1	NULL	7000.00
2	10	4400.00
3	20	19000.00
4	30	24900.00
5	40	6500.00
6	50	156400.00
7	60	28800.00
8	70	10000.00
9	80	304500.00
10	90	58000.00
11	100	51600.00
12	110	20300.00

```
select
    distinct department_id,
    sum(salary) over (partition by department_id) suma
from employees
```


WINDOW declaration

	hire_date	salary	grow
1	1987-06-17	24000.00	28400.00
2	1987-09-17	4400.00	45400.00
3	1989-09-21	17000.00	30400.00
4	1990-01-03	9000.00	32000.00
5	1991-05-21	6000.00	32000.00
6	1993-01-13	17000.00	29500.00
7	1994-06-07	6500.00	33500.00
8	1994-06-07	10000.00	28500.00

```
select
hire_date,
salary,
sum(salary) over (order by hire_date
                   ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) grow
from employees
```

WINDOW declaration

	hire_date	salary	suma_kroczaca
1	1987-06-17	24000.00	24000.00
2	1987-09-17	4400.00	28400.00
3	1989-09-21	17000.00	45400.00
4	1990-01-03	9000.00	54400.00
5	1991-05-21	6000.00	60400.00
6	1993-01-13	17000.00	77400.00
7	1994-06-07	6500.00	83900.00
8	1994-06-07	10000.00	93900.00
9	1994-06-07	12000.00	105900.00

```
select
hire_date,
salary,
sum(salary) over (order by hire_date
                   ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) suma_kroczaca
from employees
```

WINDOW declaration

	hire_date	salary	grow
1	1987-06-17	24000.00	NULL
2	1987-09-17	4400.00	24000.00
3	1989-09-21	17000.00	28400.00
4	1990-01-03	9000.00	21400.00
5	1991-05-21	6000.00	26000.00
6	1993-01-13	17000.00	15000.00
7	1994-06-07	6500.00	23000.00
8	1994-06-07	10000.00	23500.00
9	1994-06-07	12000.00	16500.00

```
select
hire_date,
salary,
sum(salary) over (order by hire_date
                   ROWS BETWEEN 2 PRECEDING AND 1 PRECEDING) grow
from employees
```

WINDOW declaration

	hire_date	salary	suma_kroczaca	suma_kroczaca
1	2011-01-13	17000.00	17000.00	17000.00
2	2012-06-07	6500.00	23500.00	53808.00
3	2012-06-07	10000.00	33500.00	53808.00
4	2012-06-07	12008.00	45508.00	53808.00
5	2012-06-07	8300.00	53808.00	53808.00
6	2012-08-16	9000.00	62808.00	62808.00
7	2012-08-17	12008.00	74816.00	74816.00
8	2012-12-07	11000.00	85816.00	85816.00

```
select
hire_date,
salary,
sum(salary) over (order by hire_date
                   ROWS BETWEEN UNBOUNDED PRECEDING
                           AND CURRENT ROW) suma_kroczaca,
sum(salary) over (order by hire_date
                   RANGE BETWEEN UNBOUNDED PRECEDING
                           AND CURRENT ROW) suma_kroczaca
from employees
```

WINDOW declaration

	department_id	okno
1	NULL	7000.00
2	10	11400.00
3	20	30400.00
4	30	55300.00
5	40	61800.00
6	50	218200.00
7	60	247000.00
8	70	257000.00
9	80	561500.00
10	90	619500.00
11	100	671100.00
12	110	691400.00

```
select
department_id,
sum(sum(salary)) over (order by department_id
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) okno
from employees
group by
department_id
```